

MODULE 15

Fundamentals for Training Deep Learning Models

Topics Covered

Lectures 1–4 — Requirements, Loss Functions & GPT Training
Lectures 5–6 — Backpropagation (Concept + Step-by-Step)
Lectures 7–8 — Gradient Descent & Types
Lectures 9–10 — Vanishing Gradient Problem & Solutions

Lectures 1–4: Requirements, Loss Functions & GPT Training

Requirements for Building a Deep Learning Model

Before writing a line of code, four ingredients must come together for a deep learning project to succeed:

Requirement	Details
Data	Labelled training data — the larger and cleaner, the better. DL typically needs 10K+ samples minimum.
Architecture	Network design — number of layers, neuron types (Dense, CNN, LSTM, Transformer), connections.
Loss Function	Measures how wrong the prediction is. The choice depends entirely on the task.
Optimiser	Algorithm that updates weights to reduce loss. Adam is the default for most tasks.

□ GPT analogy

GPT is trained on hundreds of billions of text tokens (data).

It uses a Transformer decoder architecture (architecture).

It minimises cross-entropy loss on next-token prediction (loss function).

Weights are updated with Adam optimiser (optimiser).

What is a Loss Function?

A loss function (also called cost function or objective function) quantifies the difference between the model's prediction $\hat{y}$ and the true label y . Training = minimising this value. The loss landscape is the 'terrain' the optimiser navigates.

Regression Loss Functions

Loss	Formula When to Use
Mean Squared Error (MSE)	$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$ — penalises large errors heavily. Default for regression.
Mean Absolute Error (MAE)	$MAE = (1/n) \sum y_i - \hat{y}_i $ — robust to outliers. Less sensitive than MSE.
Huber Loss	Combines MSE (small errors) + MAE (large errors). Best when dataset has outliers.
Root Mean Squared Error (RMSE)	$RMSE = \sqrt{MSE}$ — same units as target. Most interpretable regression metric.

```
import tensorflow as tf
import numpy as np

y_true = np.array([3.0, 5.0, 2.5, 7.0])
y_pred = np.array([2.8, 5.2, 3.0, 6.5])

mse = tf.keras.losses.MeanSquaredError()(y_true, y_pred).numpy()
mae = tf.keras.losses.MeanAbsoluteError()(y_true, y_pred).numpy()
hub = tf.keras.losses.Huber()(y_true, y_pred).numpy()

print(f'MSE: {mse:.4f}')    # 0.1575
```

Loss	Formula When to Use
<pre>print(f'MAE: {mae:.4f}') # 0.3500 print(f'Huber:{hub:.4f}') # 0.0738</pre>	

Binary Cross-Entropy Loss

Used for binary classification (2 classes). Measures the difference between the predicted probability and the true label:

```
# Formula: BCE = -[y · log(ŷ) + (1-y) · log(1-ŷ)]

# If true label y=1: loss = -log(ŷ)    - punish low confidence on positives
# If true label y=0: loss = -log(1-ŷ)  - punish high confidence on negatives

# Example
y_true = [1, 0, 1, 1]
y_pred = [0.9, 0.1, 0.8, 0.4] # predicted probabilities (sigmoid output)

bce = tf.keras.losses.BinaryCrossentropy()(y_true, y_pred).numpy()
print(f'BCE Loss: {bce:.4f}') # 0.2877

# In model compile:
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

GPT Training Loss — Next-Token Prediction

GPT-like models are trained on a deceptively simple task: predict the next token given all previous tokens. The loss is categorical cross-entropy over the vocabulary.

Concept	Details
Task	Given 'The cat sat on the', predict 'mat'
Output	Probability distribution over entire vocabulary (50,000+ tokens)
Loss	Cross-entropy between predicted distribution and true next token
Teacher forcing	During training, always feed the TRUE previous token (not predicted)
Perplexity	Perplexity = e^{loss} — how 'surprised' the model is. Lower = better. GPT-4 ≈ 5–15

Multi-Class Classification Loss

When the target has more than 2 classes, use categorical cross-entropy:

```
# Categorical Cross-Entropy
# L = -Σ y_i · log(ŷ_i) where sum is over all classes

# Two variants in Keras:

# 1. sparse_categorical_crossentropy - labels are integers (0, 1, 2, ...)
model.compile(loss='sparse_categorical_crossentropy')
# y_true = [0, 2, 1, 2]

# 2. categorical_crossentropy - labels are one-hot encoded
model.compile(loss='categorical_crossentropy')
# y_true = [[1,0,0], [0,0,1], [0,1,0], [0,0,1]]
```

```
# Output layer: always use softmax for multi-class
# layers.Dense(n_classes, activation='softmax')
```

Task	Output Activation Loss Function
Regression	Linear (none) → MSE / MAE / Huber
Binary classification	Sigmoid → binary_crossentropy
Multi-class classification	Softmax → sparse_categorical_crossentropy
Multi-label classification	Sigmoid → binary_crossentropy (per label)
Language modelling (GPT)	Softmax (vocab) → sparse_categorical_crossentropy

Lectures 5–6: Backpropagation

What is Backpropagation?

Backpropagation (backprop) is the algorithm used to compute gradients of the loss with respect to every weight in the network. It uses the chain rule of calculus to propagate errors backward from the output layer to the input layer, enabling the optimiser to update all weights efficiently.

□ GPT training connection

When ChatGPT is trained, after each forward pass predicting the next token, backprop computes how much each of the billions of weights contributed to the prediction error.

This runs millions of times per day across thousands of GPUs — backprop at massive scale.

Chain Rule — The Mathematical Foundation

The chain rule states that the derivative of a composite function is the product of the derivatives of each function in the chain:

```
# If y = f(g(x)), then dy/dx = (dy/dg) × (dg/dx)

# In a neural network: Loss = L(a^2(z^2(a^1(z^1(W^1, X))))
# To find ∂L/∂W^1, apply chain rule backward through every layer:

# ∂L/∂W^1 = (∂L/∂a^2) × (∂a^2/∂z^2) × (∂z^2/∂a^1) × (∂a^1/∂z^1) × (∂z^1/∂W^1)
#           ↑ output grad  ↑ activation  ↑ weight    ↑ activation  ↑ input
```

Step-by-Step Backpropagation

Step	Operation Formula
1. Forward pass	Compute $z^l = W^l a^{l-1} + b^l$ and $a^l = \text{activation}(z^l)$ for every layer l
2. Compute loss	$L = \text{loss}(y, a^L)$ where L = output layer
3. Output layer gradient	$\delta^L = \partial L / \partial a^L \times \text{activation}'(z^L)$
4. Hidden layer gradient	$\delta^l = (W^{l+1})^T \cdot \delta^{l+1} \times \text{activation}'(z^l)$
5. Weight gradient	$\partial L / \partial W^l = \delta^l \cdot (a^{l-1})^T$
6. Bias gradient	$\partial L / \partial b^l = \delta^l$

Step	Operation Formula
7. Update weights	$W^l \leftarrow W^l - \alpha \times \partial L / \partial W^l$

Backpropagation in Code (Manual)

```
import numpy as np

# Simple 2-layer network: input(2) → hidden(3) → output(1)
np.random.seed(0)
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float) # XOR
y = np.array([[0],[1],[1],[0]], dtype=float)

W1 = np.random.randn(2, 3) * 0.1
b1 = np.zeros((1, 3))
W2 = np.random.randn(3, 1) * 0.1
b2 = np.zeros((1, 1))

def sigmoid(z): return 1 / (1 + np.exp(-z))
def sigmoid_d(z): return sigmoid(z) * (1 - sigmoid(z))

lr = 0.5
for epoch in range(5000):
    # — FORWARD PASS —
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    a2 = sigmoid(z2)          # predictions

    loss = np.mean((y - a2) ** 2)

    # — BACKWARD PASS —
    dL_da2 = -2 * (y - a2) / len(y)      # ∂L/∂a2
    da2_dz2 = sigmoid_d(z2)              # ∂a2/∂z2
    delta2 = dL_da2 * da2_dz2            # output delta

    dL_da1 = delta2 @ W2.T                # ∂L/∂a1
    delta1 = dL_da1 * sigmoid_d(z1)       # hidden delta

    # — UPDATE WEIGHTS —
    W2 -= lr * (a1.T @ delta2)
    b2 -= lr * delta2.sum(axis=0, keepdims=True)
    W1 -= lr * (X.T @ delta1)
    b1 -= lr * delta1.sum(axis=0, keepdims=True)

    if epoch % 1000 == 0:
        print(f'Epoch {epoch:4d} Loss: {loss:.4f}')

print('Predictions:', a2.round())
```

□ Autograd in frameworks

In TensorFlow/PyTorch, backprop is handled automatically via automatic differentiation. PyTorch: `loss.backward()` computes all gradients. TensorFlow: `GradientTape` records operations. You never implement backprop manually in practice — understanding it conceptually is what matters.

Lectures 7–8: Gradient Descent & Types

What is Gradient Descent?

Gradient descent is the optimisation algorithm that updates network weights to minimise the loss function. It uses the gradient (slope) of the loss landscape to determine which direction to step, always moving downhill towards a minimum.

□ Mountain analogy

Imagine standing on a foggy mountain and wanting to reach the valley (minimum loss).

You feel the slope under your feet (gradient) and take a step downhill.

Learning rate = how big a step you take. Too large → overshoot; too small → takes forever.

The Update Rule

```
# Core gradient descent update:
#  $W = W - \alpha \times \partial L / \partial W$ 

# W = current weight
#  $\alpha$  = learning rate (e.g. 0.001)
#  $\partial L / \partial W$  = gradient of loss with respect to W

# If gradient is positive → W decreases (loss was rising with W)
# If gradient is negative → W increases (loss was falling with W)
# Goal: reach  $\partial L / \partial W = 0$  (flat point = minimum)
```

Types of Gradient Descent

Type	Batch Size Update Frequency Pros & Cons
Batch GD (BGD)	All N samples Once per epoch Stable, accurate gradient BUT slow, needs full data in memory
Stochastic GD (SGD)	1 sample N times per epoch Fast, noisy updates — can escape local minima BUT unstable loss curve
Mini-Batch GD	B samples (32–256) N/B times per epoch Best of both — stable + fast. Standard in deep learning

```
# Mini-batch gradient descent in Keras
model.fit(X_train, y_train,
          batch_size=32, # process 32 samples per update
          epochs=100)

# Manual mini-batch loop in NumPy
batch_size = 32
n_batches = len(X) // batch_size

for epoch in range(100):
    # Shuffle data each epoch
    idx = np.random.permutation(len(X))
    X_shuf, y_shuf = X[idx], y[idx]

    for i in range(n_batches):
        X_batch = X_shuf[i*batch_size : (i+1)*batch_size]
        y_batch = y_shuf[i*batch_size : (i+1)*batch_size]
        # forward + backward + update on this batch only
```

Effect of Batch Size

Batch Size	Effect on Training
Very small (1–8)	Noisy gradients, slow convergence, can escape sharp minima, poor GPU utilisation
Medium (32–128)	Good balance — recommended default. Stable updates, good GPU utilisation.
Large (512–4096)	Smooth gradient, fast per-epoch BUT tends to converge to sharp (worse) minima. Needs LR scaling.

Advanced Optimisers

Optimiser	Key Idea Best For
SGD + Momentum	Accumulates velocity in gradient direction — fewer oscillations. Good for CV tasks.
AdaGrad	Adapts LR per parameter — larger steps for rare features. Good for sparse data / NLP.
RMSprop	Divides LR by moving average of squared gradients — fixes AdaGrad's shrinking LR problem.
Adam	Momentum + RMSprop combined. Adaptive LR + momentum. Default for most deep learning.
AdamW	Adam + weight decay (L2 regularisation decoupled). Used to train GPT, BERT, LLaMA.

```
from tensorflow.keras.optimizers import Adam, SGD, RMSprop, AdamW

# Adam — most common default
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

# SGD with momentum
model.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9))

# AdamW — used for LLMs
model.compile(optimizer=AdamW(learning_rate=3e-4, weight_decay=0.01))

# Learning rate scheduling
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=1e-3, decay_steps=10000)
model.compile(optimizer=Adam(learning_rate=lr_schedule))
```

Lectures 9–10: Vanishing Gradient Problem

What is the Vanishing Gradient Problem?

During backpropagation, gradients are multiplied layer by layer via the chain rule. If gradients are small numbers (< 1), repeated multiplication makes them exponentially smaller as we go deeper. By the time gradients reach the early layers, they are near zero — those weights barely update, and the network fails to learn long-range dependencies.

☐ Why this matters

In a 10-layer network: if each layer shrinks the gradient by 0.5, the first layer receives $0.5^{10} \approx 0.001$ of the original gradient.

In a 50-layer network or an RNN processing 100 time steps: nearly zero. The early layers are frozen.

This was the main reason deep networks and RNNs were impractical before 2010.

Root Causes

Cause	Why It Creates Vanishing Gradients
Sigmoid / Tanh activation	Derivative of sigmoid: $\sigma'(z) = \sigma(z)(1-\sigma(z)) \leq 0.25$ always. Multiplied across 10 layers $\rightarrow$ tiny.
Poor weight initialisation	Small initial weights $\rightarrow$ small activations $\rightarrow$ small gradients from the start.
Very deep networks	Even modest per-layer shrinkage compounds exponentially across many layers.
Long sequences in RNNs	RNNs process one step at a time — gradient must flow backward through 100+ steps.

```
import numpy as np
import matplotlib.pyplot as plt

# Demonstrate gradient vanishing with sigmoid
def sigmoid(z):    return 1 / (1 + np.exp(-z))
def sigmoid_d(z):  return sigmoid(z) * (1 - sigmoid(z))

# Max derivative of sigmoid is 0.25 (at z=0)
z = np.linspace(-5, 5, 200)
plt.plot(z, sigmoid_d(z))
plt.axhline(0.25, color='red', linestyle='--', label='Max = 0.25')
plt.title('Sigmoid Derivative - Always ≤ 0.25')
plt.legend(); plt.show()

# Gradient after 20 sigmoid layers
grad = 1.0
for layer in range(20):
    grad *= 0.25          # worst-case sigmoid derivative
print(f'Gradient after 20 layers: {grad:.10f}')  # ≈ 0.0000000001
```

How to Identify Vanishing Gradients

Symptom	How to Check
Early layers not learning	Plot weight histograms per layer — early layers barely change across epochs
Loss stagnates very early	Training loss plateaus after just a few epochs
Near-zero gradients	Print gradient norms: if first layers have <code>grad_norm</code> $\approx 1e-8$, it's vanishing
RNN forgets early inputs	Model accuracy drops sharply on long sequences

```
# Monitor gradient norms in TensorFlow
import tensorflow as tf

with tf.GradientTape() as tape:
    y_pred = model(X_batch, training=True)
    loss    = loss_fn(y_batch, y_pred)
```

Symptom	How to Check
	<pre> grads = tape.gradient(loss, model.trainable_variables) for var, grad in zip(model.trainable_variables, grads): if grad is not None: print(f'{var.name:40s} grad norm: {tf.norm(grad).numpy():.6f}') # Early layers with norm < 1e-5 indicate vanishing gradients </pre>

Solutions to Vanishing Gradients

Solution	How It Helps
ReLU activation	ReLU derivative = 1 for $z > 0$, 0 for $z < 0$. No saturation → gradient flows freely through positive activations.
Leaky ReLU / ELU	Fixes the 'dying ReLU' problem (neurons stuck at 0). Small gradient even for negative inputs.
Weight initialisation	He init (for ReLU): $W \sim N(0, \sqrt{2/n})$. Xavier/Glorot (for tanh/sigmoid): $W \sim N(0, \sqrt{1/n})$.
Batch Normalisation	Normalises layer inputs to mean $\approx$ 0, std $\approx$ 1. Keeps activations in the non-saturating region.
Residual connections	Skip connections (ResNet): gradient has a direct highway to early layers without decaying.
LSTM / GRU	Designed with gates that allow gradients to flow unchanged — solves vanishing in RNNs.
Gradient clipping	Caps gradients at a max norm (e.g. 1.0) — primarily for exploding gradients but stabilises training.

```

from tensorflow.keras import layers
from tensorflow.keras.initializers import HeNormal, GlorotUniform

# — Solution 1: ReLU + He Initialisation —
layers.Dense(128, activation='relu', kernel_initializer='he_normal')

# — Solution 2: Batch Normalisation —
# Add after Dense, before activation
model = tf.keras.Sequential([
    layers.Dense(128, kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dense(64, kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dense(1, activation='sigmoid')
])

# — Solution 3: Residual Connection (skip connection) —
inputs = tf.keras.Input(shape=(128,))
x = layers.Dense(128, activation='relu')(inputs)
x = layers.Dense(128)(x)
x = layers.Add()([x, inputs])  # skip connection
x = layers.Activation('relu')(x)
outputs = layers.Dense(10, activation='softmax')(x)

# — Solution 4: Gradient Clipping —
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001,
                                     clipnorm=1.0)  # clip gradient norm to 1.0

```

□ **Vanishing vs Exploding Gradients**

Vanishing: gradients $\rightarrow 0$. Early layers freeze. Fix: ReLU, BatchNorm, ResNet.

Exploding: gradients $\rightarrow \infty$. Loss becomes NaN. Fix: gradient clipping, weight regularisation.

Both are diagnosed by monitoring gradient norms during training.

End of Module 15 Notes — Generative AI Course